performance_analysis_p1

Team ID: 30

Algorithm: Inverse Document Frequency (IDF) using MapReduce

1. Data Processing Estimation

We estimated the maximum processing capacity of the solution within the 10-minute (600 seconds) walltime limit based on the measured throughput of the largest profiled dataset (100 MB).

1.1 Throughput Calculation

Based on the profiling of the **100 MB** dataset:

- **Total Execution Time (T_{100}):** 4.57 seconds
  - *(Sum of Init: 2.00s, Map: 0.67s, Shuffle: 1.44s, Reduce: 0.46s)*
- **Input Size:** 100 MB
- **Effective System Throughput (R):**

$$R = \frac{100 \text{ MB}}{4.57 \text{ s}} \approx \mathbf{21.9} \text{ MB/s}$$

1.2 Max Data Estimation (10-minute limit)

Using the linear throughput model:

- **Time Budget:** 600 seconds
- **Calculation:**

$$D_{max} = R \times 600 \text{ s} = 21.9 \text{ MB/s} \times 600 \text{ s} = \mathbf{13,140} \text{ MB} \approx \mathbf{13.1} \text{ GB}$$

Conclusion: The system can process approximately **13 GB** of text data within the 10-minute limit. The primary bottleneck is the **Task Initialization** (splitting files) and **Shuffle** phases, which consume $\approx 75\%$ of the total runtime.

2. Profiling Results

We measured the actual execution times for three distinct input sizes to analyze scaling behavior.

2.1 Execution Time Breakdown (Seconds)

```
cat slurm-58305.out

========================================
SLURM_JOB_ID = 58305
SLURM_NODELIST = node[06-07]
SLURM_JOB_GPUS =
========================================
--- PROFILING data_1MB.txt ---
Init:    .333172153
Map:     .175003417
Shuffle: .017189427
Reduce:  .007374992
--- PROFILING data_10MB.txt ---
Init:    .318217670
Map:     .222365721
Shuffle: .151269420
Reduce:  .048935151
--- PROFILING data_100MB.txt ---
Init:    2.004315705
Map:     .667285252
Shuffle: 1.440214124
Reduce:  .457460307
```

Stage	1 MB Input	10 MB Input	100 MB Input
Task Initialization	0.333 s	0.318 s	2.004 s
Mapper	0.175 s	0.222 s	0.667 s
Shuffle / Sort	0.017 s	0.151 s	1.440 s
Reducer	0.007 s	0.049 s	0.457 s
Total Job Time	**0.532 s**	**0.740 s**	**4.568 s**

2.2 Scaling Analysis

- **Initialization (Split Overhead):** The initialization time jumped significantly from 0.3s (10MB) to 2.0s (100MB). This indicates that the `split` command is I/O intensive, as it must read the entire 100MB file to partition it line-by-line.

- **Mapper:** The Mapper is extremely efficient, taking only 0.67s to process 100MB. This suggests a raw map throughput of $\approx 150\ \mathrm{MB/s}$ (excluding overhead). The scaling is sub-linear because the fixed overhead dominates at smaller sizes.
- **Shuffle/Sort:** This phase scales **linearly** ($0.15s \rightarrow 1.44s$ for a $10\times$ data increase). This is expected as the `sort` command complexity is dominated by I/O for this data size.
- **Reducer:** Also scales **linearly** ($0.05s \rightarrow 0.46s$), maintaining a consistent processing rate.

---

3. FLOPs & Complexity Analysis

Since IDF is a text-processing algorithm, we analyze **Integer Operations (IOPs)** and **String Operations** rather than floating-point math.

3.1 Computational Complexity

Let D be the total data size (bytes) and V be the vocabulary size.

1. **Map Phase:** $O(D)$. The mapper parses every byte of the input text once.
   - *Operations:* String splitting, tokenization, hash map insertions.
2. **Shuffle Phase:** $O(K \log K)$, where K is the number of tokens emitted.
   - *Operations:* External merge sort comparisons.
3. **Reduce Phase:** $O(V)$. The reducer iterates over unique words.
   - *FLOPs:* The specific formula $IDF = \log(\frac{N}{df})$ requires **1 Division** and **1 Logarithm** per unique word.

3.2 FLOPs Estimation

- **Unique Words (V):** Estimated at ~20,000 for the 100MB dataset.
- **Total FLOPs:** $2 \times V \approx 40,000$ FLOPs.
- **Observation:** The FLOP count is negligible. The performance is dominated by **Integer Ops** (string manipulation) and **Disk I/O** (Initialization and Shuffle).

3.3 Measured Performance vs Theoretical

- **Theoretical Expectation:** $T_{total} \propto D$ (Linear).
- **Measured Reality:**
  - The jump from 10MB to 100MB ($0.74s \rightarrow 4.57s$) is a $6.1\times$ **increase** for a $10\times$ **data increase**.
  - This indicates that for 100MB, we are finally overcoming the fixed system startup overhead seen in the smaller cases. We expect the scaling to become purely linear ($10\times$) as data grows beyond 1GB.